

클라우드 인프라 교육 · 3일차 오후

AIOps — AI 시대의 인프라 운영

AI가 인프라를 만들고, 지키고, 운영하는 방식의 변화

제1장 AIOps 개관 — AI가 인프라 관리를 돕는 4가지 축

제2장 AI 워크로드를 위한 인프라 (GPU·스토리지·네트워크·아키텍처)

제3장 모니터링·로깅 × AI (Prometheus·Grafana → AI 이상탐지)

제4장 클라우드 보안 × AI (이상 트래픽·취약점 스캔)

제5장 AI 코파일럿 × IaC (Amazon Q·Copilot)

오늘 오후의 목표

- **큰 그림** — AI와 인프라의 관계는 두 방향이다: "인프라를 위한 AI(AIOps)"와 "AI를 위한 인프라". 오늘 둘 다 본다.
- **트렌드 이해** — 모니터링·보안·설계·코드 작성 각 영역에서 AI가 실제로 어떻게 쓰이는지 사례 중심으로 소개한다.
- **연결** — 이틀 반 동안 배운 모든 것(EC2·로그·보안그룹·Terraform)이 AI와 만나는 지점을 확인한다.
- **방식** — 이론·소개 중심 — 직접 해 볼 거리는 [참고] 슬라이드로 가져간다.

제1장

AIOps 개관

AI가 인프라 관리를 돕는 4가지 축

1.1 AIOps란

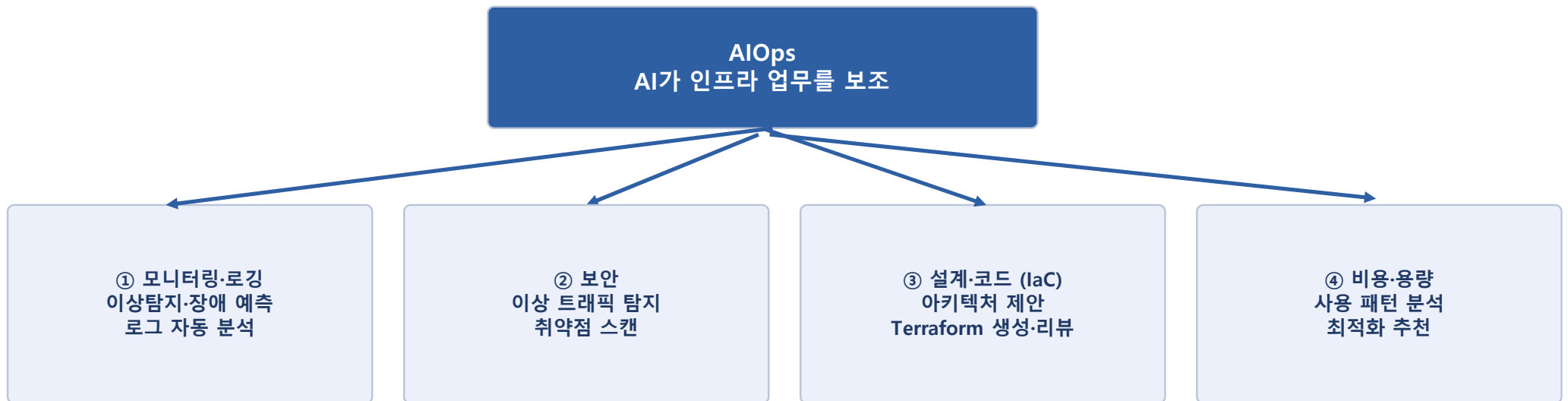
**AI for IT Operations —
인프라의 구축·운영·보안 업무에 AI를 적용해
사람의 판단과 대응을 보조·자동화하는 흐름**

- 원래는 "운영 데이터 분석"의 좁은 의미였으나, 지금은 코드 작성·설계 보조까지 넓게 통칭
- 핵심 전제: 인프라의 모든 것이 데이터(지표·로그)와 코드(IaC)가 되었기에 가능해진 일

1.2 왜 지금 AIOps인가 — 복잡도가 사람을 넘어섰다

- **인프라의 폭발** — 서버 몇 대 시대 → VM·컨테이너 수백 개, MSA 서비스 수십 개, 멀티클라우드 (이번 과정에서 본 그 진화)
- **데이터의 폭발** — 초 단위 지표 × 수백 자원 × 24시간 — 사람이 대시보드를 "보고 있는" 것으로는 커버 불가
- **속도의 요구** — 장애 1분이 곧 매출·신뢰 손실 — 감지에서 대응까지를 기계 속도로
- **인력의 한계** — 24시간 교대 관제의 비용·피로, 숙련 인력 부족 — AI가 1차 필터·보조 역할

1.3 AIOps의 4가지 축 — 오늘의 지도



네 축 모두 원리는 하나 — "인프라가 데이터·코드가 되었으니, 데이터·코드를 다루는 AI가 개입할 수 있다"

1.4 AIOps 성숙도 3단계 — 어디까지 말길 것인가

단계	내용	현재 위치
1. 관찰·분석 보조	이상 감지, 로그 요약, 원인 후보 제시 — 판단은 사람	대부분 조직의 출발점 (충분히 성숙)
2. 추천	대응 방안·코드 수정안을 제안 — 승인은 사람	빠르게 확산 중 (코파일럿·Q 등)
3. 자동 대응	정해진 범위 내 자동 복구·차단 실행	제한적 (오토스케일·자동 재시작 수준)

"AI가 다 해준다"가 아니라 단계적으로 신뢰를 쌓는 여정 — 사람의 검증 지점을 어디 둘지가 설계의 핵심

1.5 균형 잡힌 시각 — 과대광고 주의

- **되는 것** — 방대한 데이터의 패턴 감지, 반복 작업 보조, 초안 생성 — 사람보다 빠르고 지치지 않음
- **안 되는 것** — 비즈니스 맥락 판단, 책임, 처음 보는 상황의 창의적 대응 — 여전히 사람의 몫
- **현실적 태도** — "대체"가 아니라 "증강" — 24시간 지치지 않는 주니어 1명의 보조를 받는다는 감각

1.6 국내외 확산 흐름 — 어디까지 왔나

- **클라우드 콘솔 내장** — AWS·Azure·GCP 모두 콘솔에 AI 어시스턴트 탑재 경쟁 — 오류 화면에서 바로 "이게 뭐야"를 묻는 경험
- **관제 도구의 AI화** — Datadog·Grafana 등 상용·오픈소스 관제 도구가 이상탐지·요약 기능을 표준 탑재
- **개발 도구의 AI화** — Copilot 계열이 개발자 일상에 정착 — IaC도 그 연장선
- **국내** — 금융·공공의 관제 고도화 사업, 통신사·SI의 AIOps 상품화 — "AI 관제"가 제안서 표준 항목이 되는 중

제2장

AI 워크로드를 위한 인프라

GPU · 스토리지 · 네트워크 · 아키텍처 배치

2.1 반대 방향의 질문 — AI를 "위한" 인프라

1장이 "AI가 인프라를 돕는" 이야기였다면,
2장은 "AI를 돌리려면 어떤 인프라가 필요한가"다

- 생성형 AI 붐 = 사실상 인프라 붐 — CSP들의 조 단위 데이터센터·자체 칩 투자 경쟁
- AI 워크로드는 컴퓨팅·스토리지·네트워크 요구가 일반 웹과 전혀 다르다 — 그 차이가 이 장의 내용

2.2 학습 vs 추론 — 요구사항이 전혀 다른 두 워크로드

학습 (Training)

- 대규모 GPU를 며칠~몇 주 집중 사용
- 배치성 — 끝나면 자원 반납
- 스토리지: 데이터를 GPU에 끊임없이 공급하는 처리량이 관건
- 네트워크: GPU 노드 간 초고속 통신 필요
- 비용: 스팟·예약으로 절감

추론 (Inference·서빙)

- 사용자 요청마다 실시간 응답
- 상시성 — 트래픽 따라 오토스케일
- 지연시간이 UX 직결 → 사용자 인접 리전
- 상대적으로 작은 GPU·전용 칩으로 충분
- 운영 비용의 대부분이 이쪽에서 발생

2.3 컴퓨팅 — GPU 인스턴스 읽는 법

- **P 계열** — 학습용 대형 GPU (p4·p5) — 시간당 수만 원대, 클러스터로 묶어 사용
- **G 계열** — 추론·그래픽용 (g5·g6) — 학습보다 저렴, 서빙의 주력
- **AWS 자체 칩** — Trainium(학습)·Inferentia(추론) — GPU 수급난·비용의 대안
- **Day1 복습 연결** — 이름 규칙(패밀리+세대+크기) 그대로 — p5.48xlarge
- **비용 감각** — 대형 GPU는 t3.micro의 수천 배 — "안 쓰면 끄기"가 여기서도 사활 문제

GPU 비용 감각 — 규모의 차이를 숫자로

인스턴스	대략의 시간당 비용감	24시간 켜두면
t3.micro (실습용)	수십 원	십원대 — 부담 없는 수준
g 계열 (추론용 GPU)	수천 원	수만~십수만 원
p 계열 (학습용 대형)	수만 원	수십~백만 원대
p 클러스터 (수십 대)	수십만 원+	"하루에 차 한 대" 감각

정확한 단가는 수시로 변함 — 자릿수 감각만 가져갈 것. 유헤 GPU 방치가 왜 사고인지가 이 표의 메시지

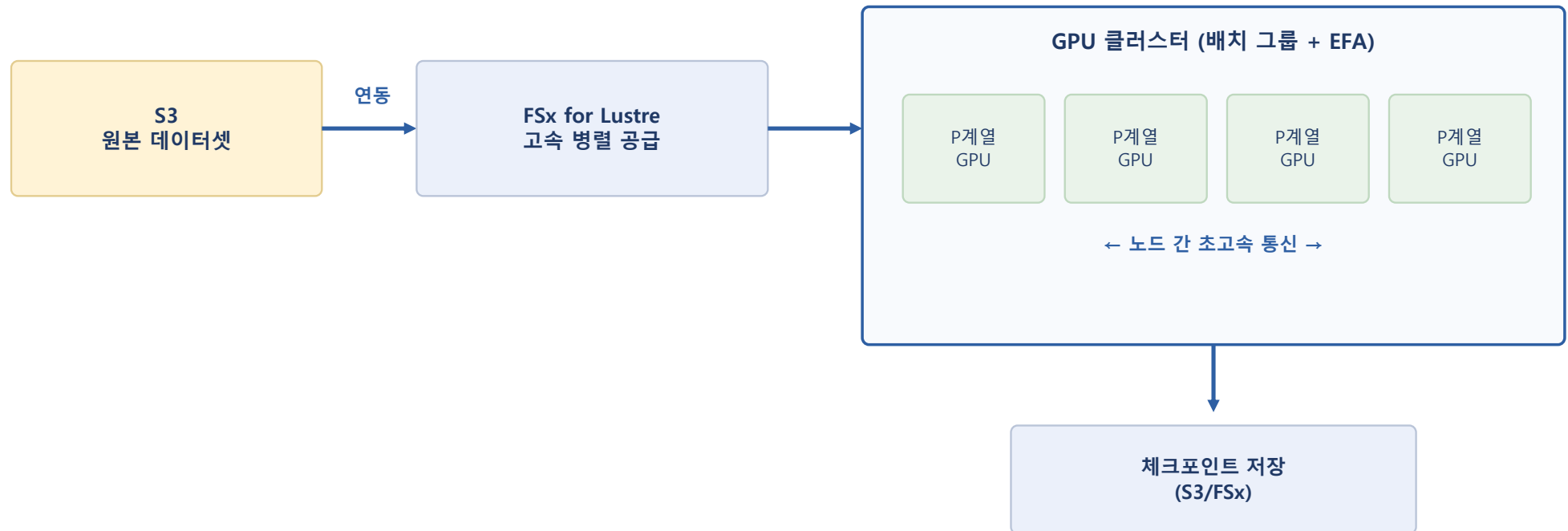
2.4 스토리지 — 학습 데이터를 어떻게 먹이는가

- **문제** — 비싼 GPU가 데이터를 기다리며 놀면 그게 최대 낭비 — 스토리지 처리량이 학습 속도를 결정
- **기본 패턴** — 원본은 S3(무제한·저렴), 학습 시 고속 파일시스템으로 공급
- **FSx for Lustre** — S3와 연동되는 초고속 병렬 파일시스템 — 학습 클러스터의 표준 조합
- **체크포인트** — 학습 중간 상태를 주기 저장(수백 GB) — 장애 시 처음부터 다시 하지 않기 위해. 스냅샷 사상의 AI판
- **Day2 연결** — EBS/S3/EFS 구분에 "고성능 병렬(FSx)"이 추가되는 그림

2.5 네트워크 — GPU들이 대화하는 속도

- **왜 중요한가** — 대형 모델은 GPU 수십~수천 개가 계산 결과를 계속 주고받으며 학습 — 네트워크가 느리면 전체가 느려짐
- **EFA** — Elastic Fabric Adapter: 노드 간 통신 지연을 극소화하는 고성능 네트워크 인터페이스
- **배치 그룹** — 인스턴스들을 물리적으로 가깝게 배치해 지연 최소화 (AZ 개념의 응용)

AI 학습 인프라 한 장 그림



공급(S3→FSx) — 계산(GPU 클러스터) — 보존(체크포인트): 학습 인프라의 3박자

2.6 추론 인프라 — 서비스가 되는 순간의 요구사항

- **지연시간** — 대화형 서비스는 사용자 인접 리전 필수 — "서울 리전에 그 모델이 있는가"가 실제 선정 기준
- **오토스케일** — 트래픽 따라 추론 서버 증감 — Auto Scaling 개념이 GPU에 적용
- **서빙 최적화** — 경량화(양자화)·배치으로 GPU 1대당 처리량 극대화 — 비용 절감의 기술 축
- **구성 스펙트럼** — 직접 GPU 서빙 ↔ 관리형 엔드포인트 ↔ API 호출

2.7 플랫폼 두 기둥 — Bedrock vs SageMaker

기준	Bedrock	SageMaker
한 줄 정의	여러 상용 모델을 하나의 API로 호출	ML 전 과정(준비~학습~배포)의 작업대
모델	남이 만든 파운데이션 모델	내가 만들거나 튜닝한 모델
과금	호출 토큰당 (서버리스형)	학습 시간 + 엔드포인트 상시 가동
시작 난이도	당일 (API 키 수준)	ML 지식·수 주 필요
적합	대부분의 기업 AI 기능	특화 모델이 경쟁력일 때

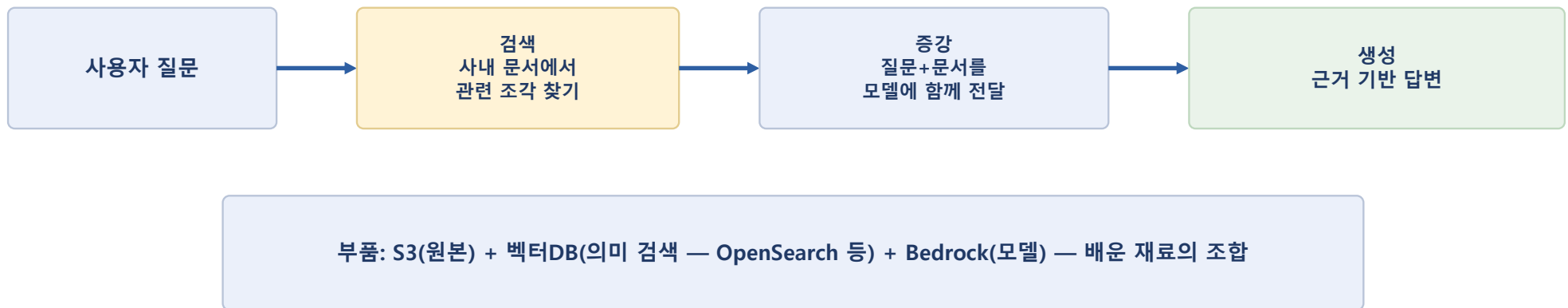
질문 하나로 구분: "모델을 만들 것인가, 쓸 것인가" — 쓸 거면 Bedrock부터

2.8 AI 도입 4가지 선택지 — 스펙트럼으로 정리



2.9 RAG — 기업 AI의 표준 패턴

재학습 없이 사내 지식을 답하게 하는 방법



"모델은 그대로, 지식은 검색으로" — 이 한 문장이면 트렌드 대화에 낄 수 있다

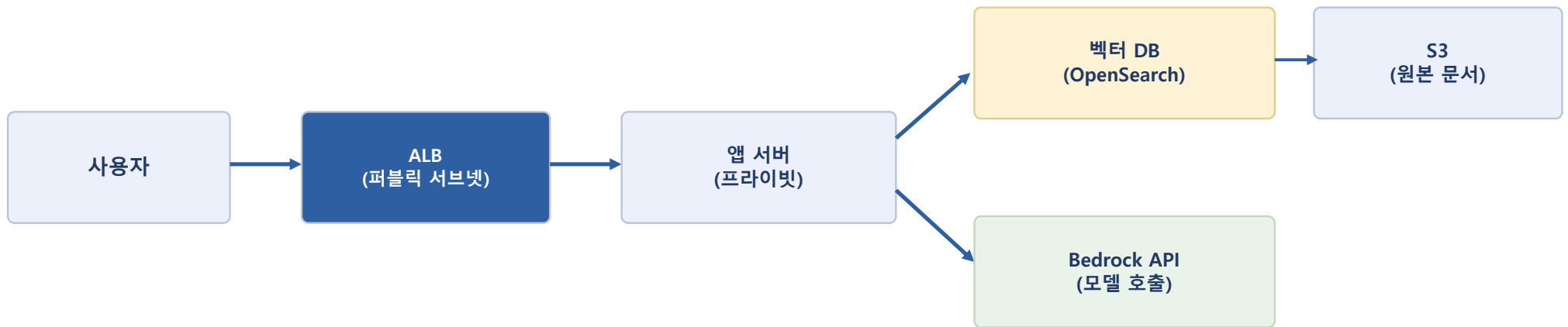
RAG의 부품 이해 — 임베딩과 벡터 DB

- **임베딩** — 문장을 "의미 좌표"(숫자 배열)로 바꾸는 것 — 뜻이 비슷하면 좌표가 가깝다
- **벡터 DB** — 그 좌표들을 저장하고 "질문과 가까운 문서"를 찾아 주는 DB (OpenSearch 등)
- **키워드 검색과의 차이** — "휴가 규정"으로 물어도 "연차 지침" 문서를 찾는다 — 단어가 아니라 의미로 검색
- **비용 주의** — 벡터 DB 상시 가동 + 문서 변경 시 임베딩 재생성 — RAG의 숨은 고정비

토큰과 컨텍스트 — AI 비용·성능의 단위

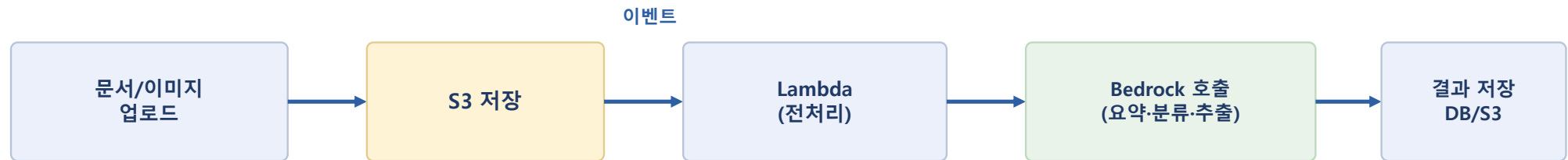
- **토큰** — 모델이 글을 세는 단위 (한글 1자 $\approx$ 1~2토큰 감각). 입력+출력 토큰만큼 과금
- **컨텍스트 창** — 한 번에 넣을 수 있는 분량의 한계 — 문서를 통째로 못 넣는 이유이자 RAG가 필요한 이유
- **비용 감각** — 짧은 질답은 수 원 미만, 긴 문서 반복 처리는 순식간에 누적 — "토큰이 곧 요금"
- **실무 습관** — 프롬프트를 짧고 정확하게 = 품질과 비용을 동시에 잡는 기술

2.10 배치 그림 ① — 사내 지식 챗봇 (표준 RAG 아키텍처)



Day1의 3계층 구조에 AI 부품 2개를 엮었을 뿐 — 새 세계가 아니라 배운 것의 확장

배치 그림 ② — 이벤트 기반 AI 처리 파이프라인



Day2 서버리스 예제(썸네일)와 같은 뼈대 — 처리 함수 자리에 AI. 서버 0대, 트래픽 0이면 비용 0

2.11 데이터 주권과 비용 — 배치의 마지막 두 기준

- **첫 질문은 데이터** — "모델을 어디 두냐"보다 "데이터가 어디를 벗어날 수 있냐" — 개인정보는 국내 리전·계정 경계 내 처리
- **비용 구조 대조** — API형=쓰는 만큼(변동비) / 자체 GPU=상시(고정비) — 호출량 적으면 API 압승, 초대량이면 역전 가능
- **숨은 비용** — 벡터 DB 상시 가동, 토큰 폭증(긴 문서), 유헤 GPU — AI판 "숨은 과금 항목"
- **소규모 조직 정석** — API 검증 → RAG → (증명되면) 파인튜닝 → 자체 GPU는 예외 — 단계적 로드맵

제3장

모니터링·로깅 × AI

Prometheus·Grafana 소개 → AI 이상탐지·예측 알림으로의 확장

3.1 모니터링의 기본 — 무엇을 보는가

- **지표(Metrics)** — 숫자의 시계열: CPU%, 메모리, 요청 수, 응답 시간 — "지금 얼마나 바쁜가"
- **로그(Logs)** — 사건의 기록: 접속, 오류, 처리 내역 — "무슨 일이 있었나"
- **추적(Traces)** — 요청 하나가 MSA 서비스들을 거쳐간 경로 — "어디서 느려졌나"
- **관측성(Observability)** — 이 셋을 합쳐 "시스템 내부를 밖에서 이해할 수 있는 능력"

3.2 도구 지형 ① — CloudWatch (AWS 기본)

- **무엇** — AWS 자원의 지표·로그·경보를 한곳에 모으는 관리형 서비스 — EC2 생성 시 기본 지표 자동 수집
- **경보(Alarm)** — "CPU 80% 5분 지속 시 알림/조치" — Auto Scaling의 방아쇠가 이 경보 (Day1 연결)
- **장점** — 설치 없음, AWS 통합 — AWS 중심이면 출발점
- **한계** — AWS 밖(온프레미스·타 클라우드)·세밀한 커스텀엔 부족 → 오픈소스 표준의 자리

도구 지형 ② — Prometheus (지표 수집의 오픈소스 표준)

- **무엇** — 서버·컨테이너의 지표를 주기적으로 긁어와(Pull) 시계열 DB에 저장하는 오픈소스
- **Exporter** — 각 대상에 붙는 "지표 창구" — node_exporter(서버) 등이 지표를 노출하면 Prometheus가 수집
- **왜 표준인가** — K8s 생태계의 사실상 기본 모니터링 — 컨테이너·MSA 시대와 함께 성장
- **PromQL** — 지표 질의 전용 언어 — "최근 5분 평균 CPU" 같은 계산식 (이름만 알아 두기)

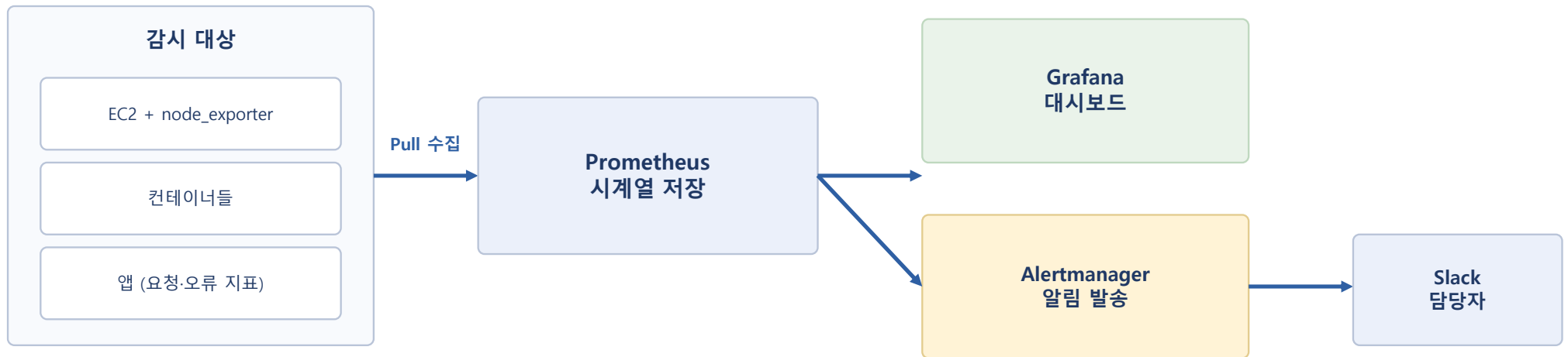
도구 지형 ③ — Grafana (보는 눈, 대시보드)

- **무엇** — 수집된 지표를 그래프·대시보드로 시각화하는 오픈소스 — "관제실 화면"의 표준
- **조합** — Prometheus(수집·저장) + Grafana(시각화) — 흔히 한 세트로 부름
- **유연성** — CloudWatch·Prometheus·DB 등 여러 소스를 한 화면에 — 하이브리드·멀티클라우드 관제에 강점
- **알림** — 조건 기반 알림 지원 — Slack·메일 연동이 일반적

Grafana 대시보드는 무엇으로 구성되나

- **패널** — 그래프 하나하나 (시계열 그래프, 게이지, 숫자, 표...) — 대시보드는 패널의 모음
- **변수** — 상단 드롭다운으로 "어느 서버/서비스를 볼지" 전환 — 대시보드 하나로 수십 대 커버
- **표준 구성 예** — 최상단: 서비스 상태 요약(신호등) / 중단: 핵심 4지표(트래픽·오류율·지연·포화도) / 하단: 상세
- **실무 팁** — 커뮤니티 대시보드를 가져와 시작하는 것이 관례 — 처음부터 만들지 않는다

모니터링 파이프라인 한 장 그림



수집(Exporter→Prometheus) — 보기(Grafana) — 알리기(Alertmanager): 표준 3종 세트

3.3 전통 방식의 한계 — 임계값과 알림 피로

- **고정 임계값의 딜레마** — "CPU 80% 넘으면 알림" — 새벽엔 30%도 이상인데 못 잡고, 이벤트 날엔 90%도 정상인데 울린다
- **알림 피로(Alert Fatigue)** — 오탐이 쌓이면 사람이 알림을 무시 — 진짜 장애를 놓치는 역설
- **로그의 바다** — 하루 수 GB 로그에서 이상 징후를 사람이 찾는 것의 한계
- **사후 대응** — 터진 뒤에 아는 구조 — "터지기 전에 알 수 없나?"가 AI 확장의 출발점

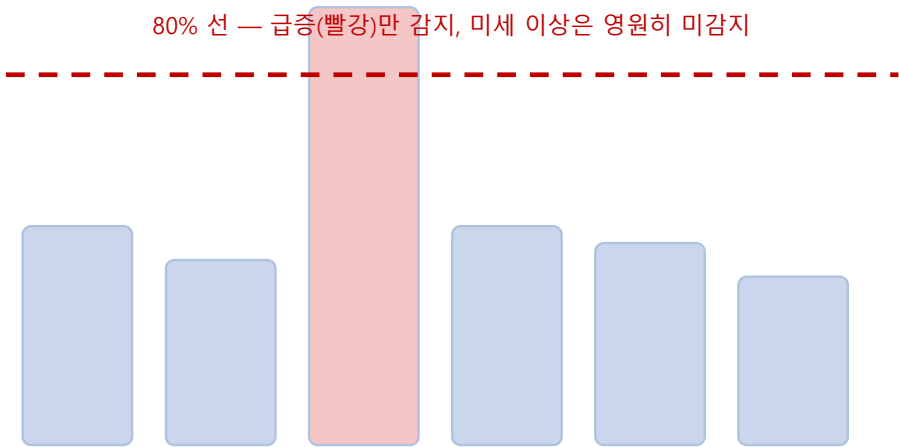
3.4 AI 확장 ① — 이상탐지 (Anomaly Detection)

고정 임계값 대신, AI가 지표의 "평소 패턴"을 학습해
패턴에서 벗어난 순간을 잡아낸다

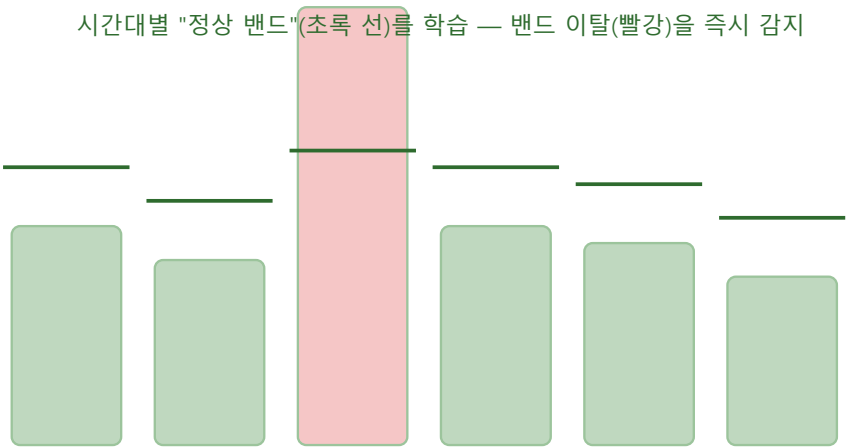
- 요일·시간대별 정상 범위를 스스로 계산 — 새벽의 미세 이상도, 이벤트 날의 정상 급증도 구분
- AWS 구현: CloudWatch Anomaly Detection — 경보 생성 시 체크 하나로 켜지는 수준까지 상용화
- Grafana 진영도 ML 기반 이상탐지 확장 중

고정 임계값 vs AI 이상탐지 — 그림으로

고정 임계값



AI 이상탐지



선이 수평이냐, 패턴을 따라 움직이냐 — 이 차이가 오탐과 미탐을 동시에 줄인다

3.5 AI 확장 ② — 예측적 알림과 장애 예방

- **개념** — "지금 이상"이 아니라 "이대로면 3시간 뒤 이상" — 추세를 학습해 미래를 경고
- **대표 사례** — 디스크 증가 추세 → 고갈 시점 예측 / 메모리 누수 조기 감지 / 트래픽 급증 예측 → 사전 증설
- **AWS 구현** — DevOps Guru: 지표·이벤트를 ML 분석해 "이상 징후+원인 후보+권장 조치"를 묶어 제안
- **효과** — 사후 대응 → 사전 예방으로의 전환 — 관제의 패러다임 변화

3.6 AI 확장 ③ — 로그를 AI가 읽는다

- **패턴 클러스터링** — 수백만 줄 로그를 유형별로 자동 묶음 — "오늘 새로 나타난 오류 유형"만 골라 보기
- **자연어 질의** — "어젯밤 결제 오류 원인 요약해 줘" — 쿼리 언어 대신 말로 로그를 조회하는 방향
- **장애 요약** — 장애 전후의 로그·이벤트·배포 이력을 AI가 타임라인으로 요약 — 원인 분석 시간 단축
- **현실 감각** — "원인 확정"이 아니라 "사람이 볼 후보를 추려 줘" — 성숙도 1~2단계의 전형

3.7 알림 설계의 원칙 — AI 이전에 지켜야 할 것

- **행동 가능성** — "받으면 뭘 해야 하는지"가 분명한 알림만 — 조치 없는 알림은 소음
- **심각도 구분** — 깨워야 할 것(전화) / 아침에 볼 것(메일) — 채널로 급을 나눈다
- **묶음과 억제** — 같은 원인의 알림 100개는 1개로 — 폭풍 방지
- **AI와의 관계** — 이 설계가 없으면 AI 이상탐지를 켜도 소음만 늘어난다 — 원칙 위에 AI

제3장 정리 — 관제의 진화 방향

보는 것(대시보드) → 알리는 것(임계값 경고)
→ 이해하는 것(AI 이상탐지·요약) → 내다보는 것(예측)

- 도구는 CloudWatch든 Prometheus+Grafana든 — 방향은 동일
- 시작은 쉬운 것부터: CloudWatch 경보에 이상탐지 옵션 켜 보기

제4장

클라우드 보안 × AI

AI 기반 이상 트래픽 탐지 · 취약점 스캔

4.1 전통 보안 운영의 한계

- **룰 기반의 태생적 한계** — "알려진 공격 패턴"만 잡는다 — 처음 보는 수법(제로데이·신종)은 룰이 없어 통과
- **양의 문제** — 방화벽·시스템 로그 하루 수백만 건 — 사람이 전수 검토 불가
- **오탐의 홍수** — 알림 피로가 보안 관제(SOC)에도 동일하게
- **과정 연결** — Day2 "퍼블릭 IP는 수 분 내 스캔당한다" — 그 공격 시도를 누가 다 보고 있나?

4.2 AI 접근 — 행동 기반 탐지

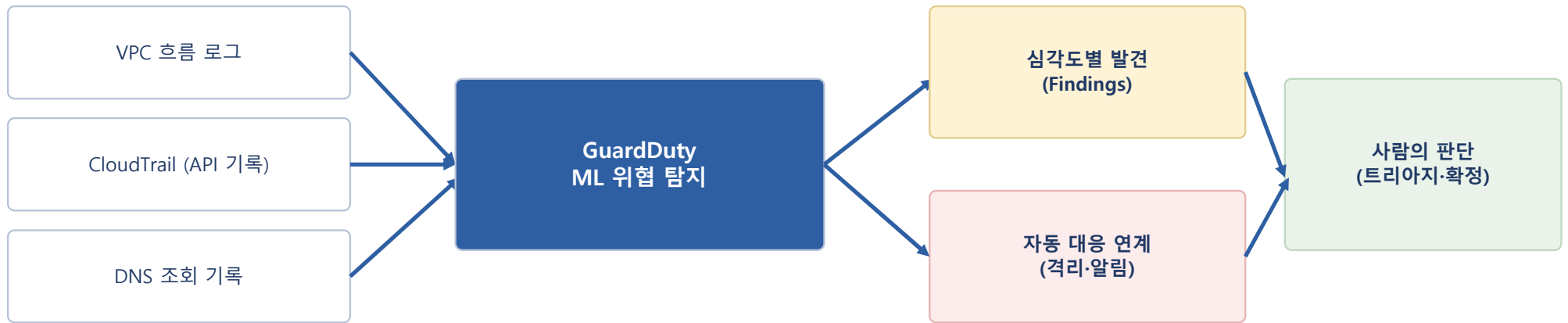
**"알려진 나쁜 패턴 목록"과 대조하는 대신,
"평소 행동"을 학습해 벗어난 행동을 잡는다**

- 3장 이상탐지와 같은 원리를 보안 데이터에 적용
- 평소 안 하던 일 — 새벽의 대량 다운로드, 처음 보는 국가에서의 API 호출, 갑작스러운 권한 변경 — 을 잡는다

4.3 AWS 구현 — GuardDuty (이상 트래픽·행위 탐지)

- **무엇** — VPC 흐름 로그·DNS 조회·CloudTrail(API 기록)을 ML로 분석하는 관리형 위협 탐지
- **잡는 것** — 코인 채굴 통신, 악성 IP 접속, 비정상 API 시퀀스, 자격증명 도용 의심
- **왜 강력한가** — 에이전트 설치 없이 켜기만 하면 동작 — AWS 전체 데이터로 학습한 모델의 혜택
- **과정 연결** — "모든 API는 기록된다"(CloudTrail) — 그 기록이 AI 탐지의 원료

AI 보안 감시의 큰 그림



기록이 원료 — AI가 1차 필터 — 사람이 최종 판단: 성숙도 단계 그대로

4.4 SOC(보안 관제)에서의 AI 활용

- **알림 트리아지** — 하루 수천 건 알림을 위험도·연관성으로 선별 — 분석가는 상위 건에 집중
- **시나리오 요약** — 흩어진 이벤트를 "하나의 공격 흐름"으로 엮어 서사로 제시 — 침투 경로 파악 단축
- **자연어 조사** — "이 IP가 지난주 접근한 자원 전부" — 질의 언어 없이 조사
- **주의** — AI 탐지를 회피하는 AI 공격도 진화 — 다층 방어의 가치는 오히려 상승

4.5 새 주제 — AI 서비스 "자체"의 보안

- **프롬프트 인젝션** — 사용자 입력으로 시스템 지시를 뒤집으려는 공격 ("이전 지시 무시하고...") — 입력 검증·권한 분리로 완화
- **데이터 유출 경로** — RAG가 권한 없는 문서까지 검색해 답하면 그게 곧 유출 — 검색 단계 권한 필터 필수
- **가드레일** — 유해·금지 주제 차단, 개인정보 마스킹 계층 (Bedrock Guardrails 등 관리형 제공)
- **원칙 불변** — 최소 권한·입력 불신·전 구간 로깅 — 제로 트러스트가 AI에도 그대로

제4장 정리

**보안의 AI화 = "목록 대조"에서 "행동 이해"로
그 원료는 3일 내내 강조한 "모든 것의 기록"**

- 커는 것부터: GuardDuty·ECR 스캔은 클릭 몇 번의 관리형
- 원칙 불변: 최소 권한·최소 개방이 잘 되어 있을수록 AI 탐지도 정확하다

제5장

AI 코파일럿 × IaC

Amazon Q · GitHub Copilot — 인프라 코드 작성의 새 표준

5.1 왜 IaC가 AI와 만나는가

오전에 인프라가 "코드(텍스트)"가 되는 것을 직접 봤다
텍스트가 된 순간, 텍스트를 다루는 AI의 사정권에 들어온 것

- Terraform은 공개 코드·문서·Q&A가 가장 풍부한 IaC — AI가 특히 잘하는 영역
- 안전장치도 이미 있다: AI가 실수해도 plan이 실행 전에 잡는다 — "AI 생성 + 도구 검증" 조합

5.2 도구 소개 — 코파일럿 지형

도구	특징	적합
Amazon Q Developer	AWS 네이티브 — 콘솔·IDE 통합, AWS 서비스·오류 맥락 이해	AWS 중심 조직
GitHub Copilot	에디터 안 실시간 자동완성 + 채팅	일반 코딩 전반
범용 챗봇 (Claude 등)	설치 없이 대화창에서 생성·리뷰·설명	학습·검토·단발 작업
에이전트형	파일 수정·명령 실행까지 위임 (확산 중)	반복 작업 자동화

도구는 달라도 활용 패턴과 검증 원칙은 동일

5.3 활용 패턴 4가지 — 현장에서 이렇게 쓰인다

패턴	활용 방식	효과
① 생성	요구사항을 조건으로 명시 → 코드 초안 생성	작성 시간 단축
② 리뷰	기존 코드를 주고 보안·모범사례 검토 요청	제3의 눈 — 가장 안전한 첫 활용
③ 디버깅	오류 메시지 전체를 붙여 원인·수정 요청	트러블슈팅 시간 단축
④ 리팩토링	변수화·모듈화 등 구조 개선 위임	품질 개선 진입장벽 하락

②(리뷰)부터 시작하는 것이 조직 도입의 정석 — 읽기 전용이라 위험이 없다

[참고] 패턴 ① 생성 — 요청문 견본

다음 조건으로 AWS Terraform 코드를 작성해 주세요.

- 리전: ap-northeast-2 (서울)
- VPC 1개 (10.100.0.0/16), 퍼블릭 서브넷 1개 (2a AZ)
- 인터넷 게이트웨이·라우팅 테이블 구성 포함
- EC2 1대: 반드시 t3.micro, 최신 Amazon Linux 2023을 data로 조회
- 보안그룹: 80 포트만 전체 개방
- 모든 자원에 Owner 태그, provider 5.x, 한국어 주석

포인트: 리전·타입·보안·버전을 "숫자로 명시"

조건이 구체적일수록 결과가 정확하다

나쁜 예: "AWS 서버 만드는 코드 줘" → 리전·타입이 제멋대로

[참고] 패턴 ②③ 리뷰·디버깅 — 요청문 견본

리뷰 요청:

아래 Terraform 코드를 보안 관점에서 검토해 주세요.
위험 설정은 심각도와 수정 코드를 함께 제시해 주세요.
(코드 붙여넣기)

디버깅 요청:

terraform plan에서 아래 오류가 났습니다. 원인과 수정 방법은?
오류: (에러 메시지 전체 - 요약하지 말 것)
관련 코드: (해당 부분)

오류 메시지를 통째로 주는 것이 요령 -

AI는 메시지 속 줄번호·자원명까지 읽는다

이틀간 손으로 하던 트러블슈팅(오류 읽기→원인 찾기)의 AI 보조판

5.4 프롬프트 4대 요령 — 견본에 다 들어있다

요령	방법	견본 속 위치
역할·관점 지정	"보안 검토자 관점에서"처럼 시각 부여	리뷰 견본의 "보안 관점"
제약 명시	리전·타입·버전 등 지켜야 할 것을 숫자로	생성 견본의 t3.micro·5.x
형식 지정	파일 구성·주석 언어·출력 형태 요구	생성 견본의 "한국어 주석"
맥락 제공	기존 코드·오류 전문을 함께 첨부	디버깅 견본의 "전체 붙여넣기"

프롬프트 엔지니어링의 90%는 이 4가지 — 나머지는 반복과 재질문

재질문 기법 — 한 번에 안 끝내는 것이 정상

- **설명 요청** — "이 블록이 왜 필요한지 설명해 줘" — 모르는 줄을 그냥 넘기지 않는 습관
- **대안 요청** — "보안그룹을 더 좁히는 대안 두 가지를 비교해 줘" — 하나의 답에 갇히지 않기
- **기준 요청** — "이 구성의 월 비용 요소를 나열해 줘" — 비용 감각 확인
- **축소 요청** — "방금 코드에서 EC2 부분만 다시" — 필요한 조각만 재생성

5.5 AI 산출물 검증 5단계 — 반드시 세트로 기억

1

읽기 — 모르는 블록이 있으면 그대로 쓰지 않는다 ("이 줄 설명해 줘"로 재질문)

2

정책 대조 — 리전·타입·보안이 조직 규칙에 맞는가

3

plan — 무엇이 생기고 교체(-/+)는 없는지 — 오전에 배운 그 견적서

4

비용 감각 — 시간당 과금 자원이 몰래 들어있지 않은가

5

작게 적용 — 테스트 환경부터. 신뢰는 반복 검증으로

5.6 한계와 주의 — 알고 쓰면 무기

- **환각** — 존재하지 않는 속성을 그럴듯하게 생성 — plan·init이 그물이지만 문서 확인 습관 필요
- **구버전 문법** — 학습 시점 탓에 예전 문법 가능 — 버전 명시 + deprecated 경고 확인
- **비밀정보** — 액세스 키·비밀번호·내부 정보를 채팅창에 붙여넣지 않는다 — 사내 규정 확인 필수
- **책임** — apply를 누른 사람이 책임진다 — AI는 보조자이지 책임 주체가 아니다

5.7 조직 도입 가이드 — 순서가 있다

- **사용 규정 먼저** — 무엇을 붙여넣어도 되는가(코드 O, 비밀·고객정보 X)를 문서로 — 개인 판단에 맡기지 않기
- **표준 프롬프트 자산화** — 검증된 요청문(오늘의 견본 같은)을 팀 위키에 축적
- **리뷰 절차 유지** — AI 생성 코드도 사람 코드와 동일한 리뷰 + plan 게이트 통과
- **확장 순서** — 리뷰(읽기 전용) → 생성 → 에이전트 — 신뢰를 단계로 쌓는다
- **교육 확장 가능성** — 오전 Terraform 실습에 이 패턴을 얹으면 "AI 협업 IaC 실습"으로 확장 가능 — 향후 과정 설계 참고

제5장 정리 — AI 시대의 인프라 역량

타이핑의 가치는 내려가고,
"정확히 요구하고, 결과를 검증하는" 능력의 가치가 올라간다

- 요구의 어휘 = 3일간 배운 개념들 (리전·CIDR·SG·타입...)
- 검증의 도구 = 오전에 익힌 plan
- 그래서 이 과정의 순서가 개념 → 손 → 코드 → AI 였던 것